

LAPORAN PRAKTIKUM
Sistem Operasi

MODUL 11
(Memori Xinu)



Universitas
Telkom

Disusun Oleh:
Ryan Gabriel Togar Simamora (2311104045)
Kelas S1SE-07-02

Dosen : Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I

PENDAHULUAN

1.1 Dasar Teori



A. Struktur Memori pada Xinu

Pada modul ini, kita akan mempelajari salah satu subsistem terpenting dalam sistem operasi, yaitu Manajemen Memori. Berbeda dengan sistem operasi modern seperti Linux yang menggunakan mekanisme Virtual Memory dan Paging, Xinu memakai pendekatan yang lebih sederhana namun efektif untuk tujuan pembelajaran: manajemen memori fisik langsung menggunakan struktur Linked List.

Fokus utama kita adalah memahami bagaimana Xinu membagi memori menjadi Stack dan Heap, serta bagaimana sistem menangani proses alokasi (getmem) dan dealokasi (freemem) memori secara dinamis. Kita akan menelusuri struktur memlist yang berfungsi sebagai catatan utama untuk memori bebas di Xinu.

Ketika Xinu dikompilasi, ia menghasilkan sebuah memory image yang tersusun secara terstruktur. Empat bagian utama di dalamnya adalah:

1. Text Segment

Bagian inilah yang memuat seluruh kode program yang sudah melalui proses kompilasi. Semua fungsi Xinu, system utilities, dan perintah shell berada di sini.

Text segment menempati alamat memori terendah.

2. Data Segment

Berisi global variables yang sudah diberi nilai awal.

Semua variabel global yang diinisiasi akan ditempatkan pada segmen ini.

3. BSS Segment

Berisi global variables tanpa inisialisasi (default bernilai 0).

Segmen ini otomatis diisi nol oleh sistem.

4. Free Space

Sisa ruang memori kosong yang nantinya dipakai Xinu ketika berjalan.

Di sinilah stack proses dan heap akan dialokasikan.

2. Apa yang Terjadi Saat Xinu Dijalankan?

Ketika proses dibuat atau dialokasikan, Xinu memanfaatkan free space untuk dua struktur penting:

A. Stack

- Dibuat saat pemanggilan create()
- Dihapus ketika proses berakhir
- Dialokasikan dari alamat tertinggi → ke bawah
- Menampung variabel lokal, return address, dan context proses lainnya

B. Heap

- Dibuat saat program memanggil getmem() (mirip malloc() di C)
- Dialokasikan dari bawah → ke atas
- Dihapus memakai freemem(), tidak otomatis hilang saat proses mati
- Jika proses lupa memanggil freemem() → terjadi memory leak

3. Fungsi-Fungsi Manajemen Memori Xinu

Xinu menyediakan empat fungsi inti untuk mengelola memori:

A. getstk()

Digunakan oleh create() untuk membuat stack baru.

Dialokasikan dari memori paling atas.

```
getstk(size);
```

B. freestk()

Menghapus stack milik proses yang sudah selesai.

```
freestk(stackaddr, size);
```

Selalu dipanggil ketika proses berakhir.

C. getmem()

Mengambil memori dari heap (dynamic allocation).

```
char *ptr = getmem(nbytes);
```

Jika memori tidak cukup → mengembalikan SYSERR.

D. freemem()

Mengembalikan memori heap ke free space.

```
freemem(ptr, size);
```

Programmer wajib memanggil fungsi ini untuk mencegah memory leak.

Tujuan Praktikum:

1. Mahasiswa mampu memahami struktur memori pada Xinu

Manfaat Praktikum :

1. Memahami Struktur Memory Pada Xinu

BAB II HASIL PRAKTIKUM

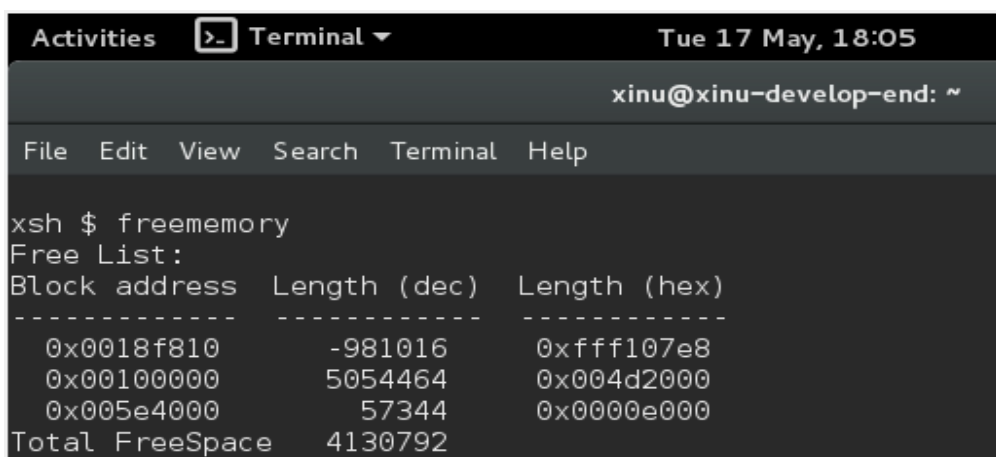
2.1 Guided Praktikum (Kosong)

2.2 Jurnal Praktikum

Jurnal

1. [80 poin] Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:
 - a. [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

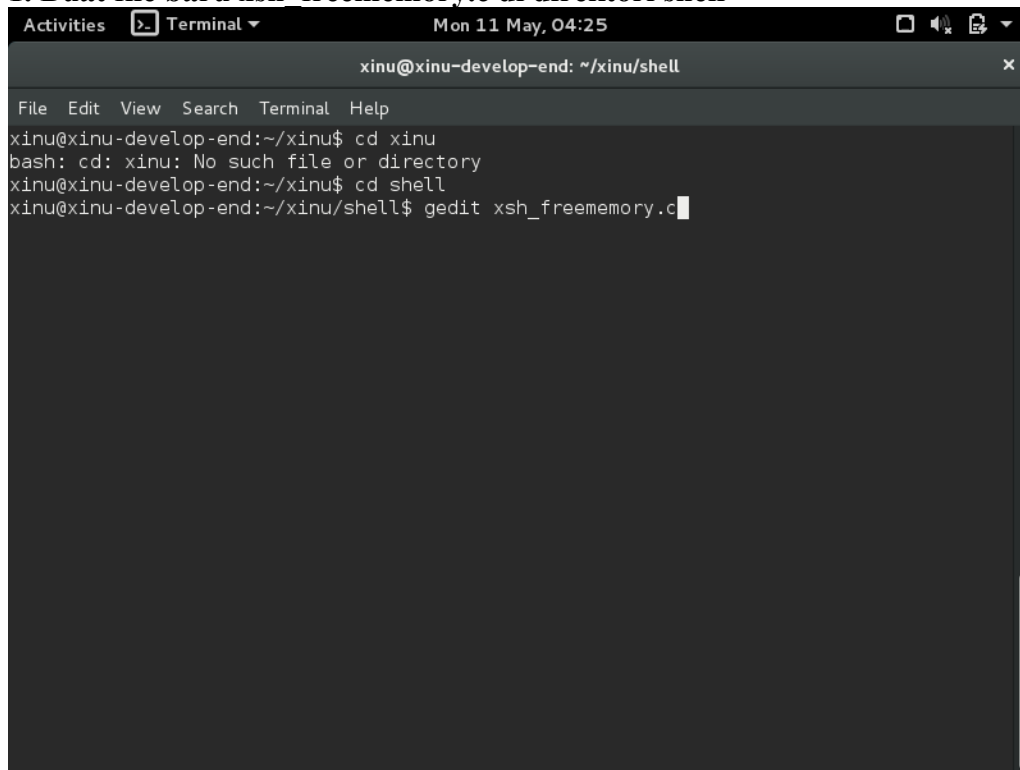
Output yang diinginkan:



```
Activities [Terminal] Tue 17 May, 18:05
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
xsh $ freememory
Free List:
Block address Length (dec) Length (hex)
-----
0x0018f810 -981016 0xffff107e8
0x00100000 5054464 0x004d2000
0x005e4000 57344 0x0000e000
Total FreeSpace 4130792
```

Jawab :

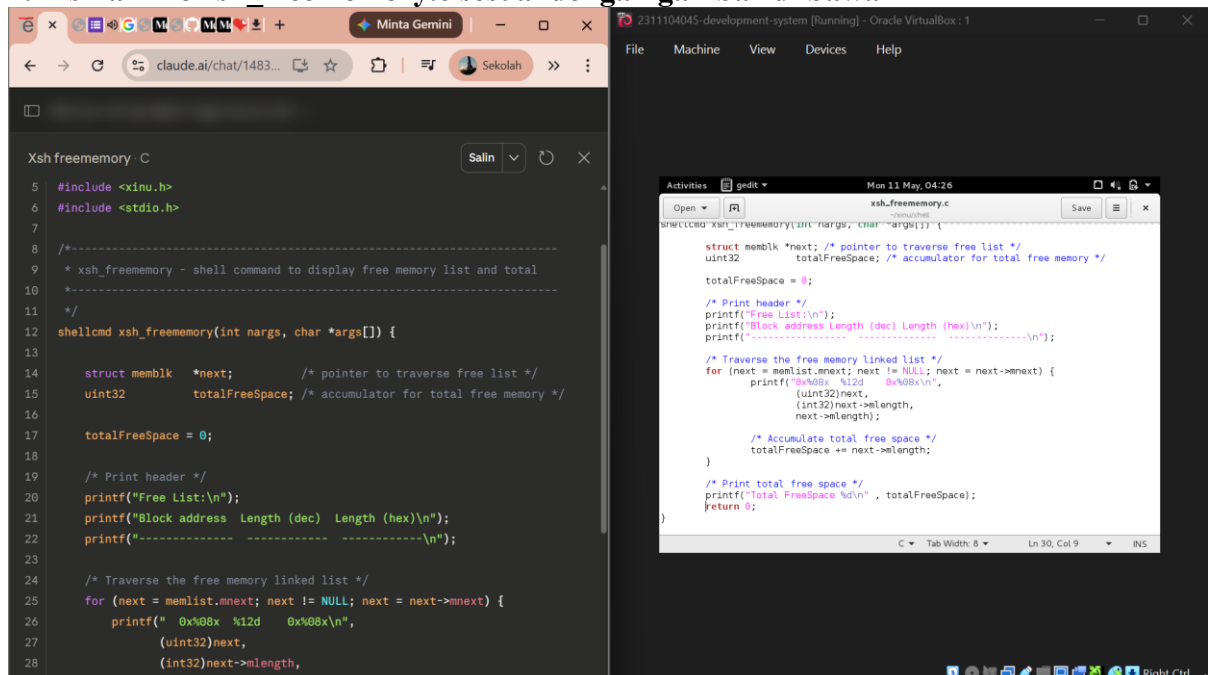
1. Buat file baru xsh_freememory.c di direktori shell



```

xinu@xinu-develop-end: ~/xinu/shell
File Edit View Search Terminal Help
xinu@xinu-develop-end:~/xinu$ cd xinu
bash: cd: xinu: No such file or directory
xinu@xinu-develop-end:~/xinu$ cd shell
xinu@xinu-develop-end:~/xinu/shell$ gedit xsh_freememory.c
  
```

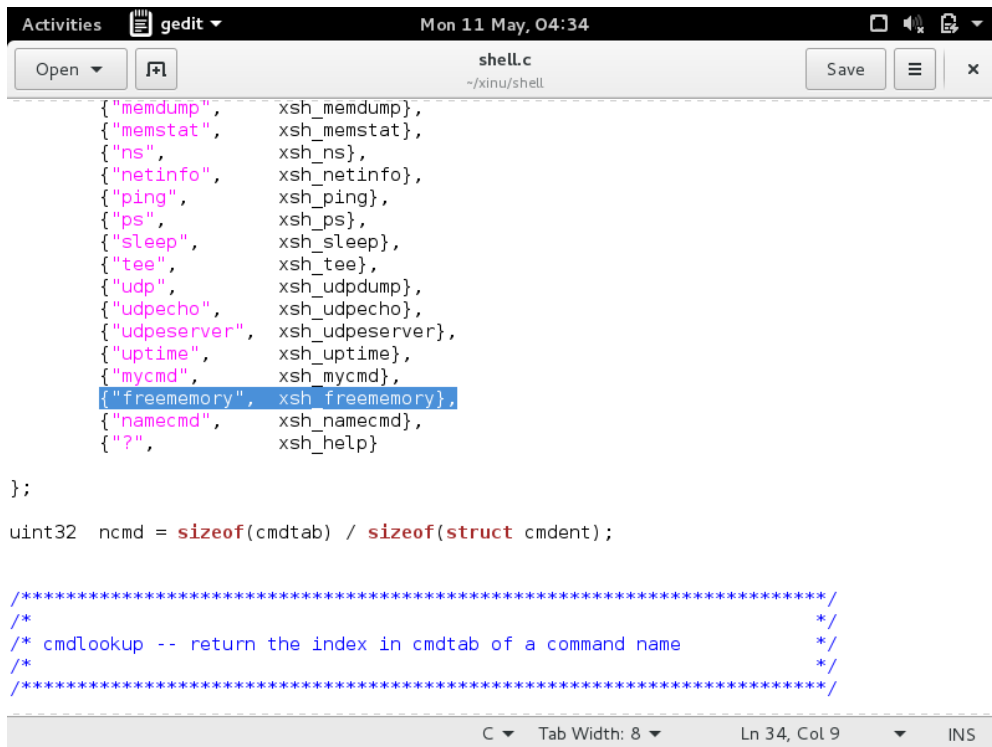
2. Isikan file xsh_freememory.c sesuai dengan gambar di bawah ini



```

Xsh freememory - C
5 #include <xinu.h>
6 #include <stdio.h>
7
8 /*-----
9  * xsh_freememory - shell command to display free memory list and total
10  *-----
11  */
12 shellcmd xsh_freememory(int nargs, char *args[]) {
13
14     struct memblk *next; /* pointer to traverse free list */
15     uint32 totalFreeSpace; /* accumulator for total free memory */
16
17     totalFreeSpace = 0;
18
19     /* Print header */
20     printf("Free List:\n");
21     printf("Block address Length (dec) Length (hex)\n");
22     printf("-----\n");
23
24     /* Traverse the free memory linked list */
25     for (next = memlist.mnext; next != NULL; next = next->mnext) {
26         printf(" 0x%08x %12d 0x%08x\n",
27             (uint32)next,
28             (int32)next->mlength,
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
    }
    /* Accumulate total free space */
    totalFreeSpace += next->mlength;
    }
    /* Print total free space */
    printf("Total FreeSpace %d\n", totalFreeSpace);
    return 0;
}
  
```

3. Setelah membuat file tersebut, kamu perlu mendaftarkan command di shell Xinu.



```

Activities  gedit  Mon 11 May, 04:34
Open  [icon]  shell.c  Save  [icon]  x
~/xinu/shell

{"mемdump",    xsh_memdump},
{"memstat",    xsh_memstat},
{"ns",         xsh_ns},
{"netinfo",    xsh_netinfo},
{"ping",       xsh_ping},
{"ps",         xsh_ps},
{"sleep",      xsh_sleep},
{"tee",        xsh_tee},
{"udp",        xsh_udpdump},
{"udpecho",    xsh_udpecho},
{"udpserver",  xsh_udpserver},
{"uptime",    xsh_uptime},
{"mycmd",      xsh_mycmd},
{"freememory", xsh_freememory},
{"namecmd",    xsh_namecmd},
{"?",         xsh_help}

};

uint32  ncmd = sizeof(cmdtab) / sizeof(struct cmdent);

/*****
/*
/* cmdlookup -- return the index in cmdtab of a command name
/*
/*
*****/
  
```



```

Activities  gedit  Mon 11 May, 04:40
Open  [icon]  shprototypes.h  Save  [icon]  x
~/xinu/include

extern shellcmd xsh_sleep (int32, char *[]);

/* in file xsh_udpdump.c */
extern shellcmd xsh_udpdump (int32, char *[]);

/* in file xsh_udpecho.c */
extern shellcmd xsh_udpecho (int32, char *[]);

/* in file xsh_udpserver.c */
extern shellcmd xsh_udpserver (int32, char *[]);

/* in file xsh_uptime.c */
extern shellcmd xsh_uptime (int32, char *[]);

/* in file xsh_mycmd.c */
extern shellcmd xsh_mycmd (int32, char *[]);

/* in file xsh_namecmd.c */
extern shellcmd xsh_namecmd (int32, char *[]);

/* in file xsh_freememory.c */
extern shellcmd xsh_freememory (int32, char *[]);

/* in file xsh_help.c */
extern shellcmd xsh_help (int32, char *[]);

/* in file xsh_tee.c */
extern shellcmd xsh_tee (int32, char *[]);
  
```

4. Lalu kita masuk kedirektori compile dan ketikkan perintah make clean dan make

```

Activities  Terminal  Mon 11 May, 04:56
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help

-00 -DBRELOC=0x150000 -DBOOTPL0C=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPL0C=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPL0C=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPL0C=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/clkdsp.o ../system/clkdsp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPL0C=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPL0C=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPL0C=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPL0C=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$

```

5. Lalu masuk ke sudo minicom dan ketikkan perintah freememory

```

Activities  Terminal  Mon 11 May, 05:01
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help

[0x0011FF60 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)

-----Nama:Ryan_Gabriel_Togar_Simamora-----

XINU

-----NIM: 2311104045-----

Welcome to Xinu!

xsh $ freememory
Free List:
Block address Length (dec) Length (hex)
-----
0x0018e110 -975128 0xffff11ee8
0x00100000 5070848 0x004d6000
0x005e8000 57344 0x0000e000
Total FreeSpace 4153064
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:1 | ttyS0

```

2. [4 poin per subsoal] Jawablah pertanyaan berikut:

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Jawab :

Xinu memisahkan keduanya karena keduanya memiliki karakteristik berbeda. Data segment menyimpan variabel global yang sudah diinisiasi nilainya, sehingga nilai tersebut harus disimpan secara eksplisit di dalam binary. BSS segment menyimpan variabel global yang belum diinisiasi, sehingga cukup dicatat ukurannya saja (tidak perlu menyimpan nilai nol secara eksplisit).

Pemisahan ini menghemat ukuran file binary karena BSS tidak perlu menyimpan data, cukup diinisiasi ke nol saat loading.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Jawab :

Setiap kali proses baru dibuat, Xinu mengalokasikan blok dari free space untuk stack proses tersebut, sehingga ukuran free space berkurang. Ketika proses terminasi, memori stack dikembalikan (dealokasi) dan free space bertambah kembali. Untuk heap, alokasi terjadi saat `getmem()` dipanggil dan dealokasi saat `freemem()` dipanggil secara manual. Jika heap tidak dibebaskan meskipun proses sudah terminasi, free space tidak bertambah, terjadi memory leak.

c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Jawab :

Stack bersifat terstruktur (LIFO) dan dealokasi terjadi otomatis saat fungsi/proses selesai, sehingga tidak ada risiko memory leak dari stack. Sebaliknya, heap dialokasikan dan didealokasi secara bebas dan manual, programmer harus secara eksplisit memanggil `freemem()`. Jika lupa, terjadi memory leak. Selain itu, heap dapat mengalami fragmentasi, yaitu kondisi di mana total free space cukup tapi tidak ada blok yang cukup besar secara berurutan untuk memenuhi permintaan alokasi baru.

d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Jawab :

Linked list dipilih karena free block tersebar di berbagai alamat memori yang tidak berurutan (akibat alokasi dan dealokasi dinamis). Dengan linked list, setiap free block menyimpan pointer ke free block berikutnya, sehingga Xinu dapat menelusuri semua blok kosong tanpa memerlukan array berukuran tetap. Struktur ini juga efisien untuk operasi insert dan delete saat blok dialokasikan atau dibebaskan, tanpa perlu memindahkan data.

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Jawab :

Ada tiga tantangan utama:

1. Tidak ada garbage collection, heap harus dibebaskan secara manual oleh programmer, dan jika proses terminasi tanpa membebaskan heap, terjadi memory leak permanen.
2. Fragmentasi, seiring berjalannya waktu, blok-blok kecil yang tersebar dapat membuat alokasi blok besar gagal meskipun total free space sebenarnya mencukupi.
3. Overflow, permintaan heap bisa melebihi sisa free space yang tersedia, menyebabkan kegagalan alokasi yang harus ditangani secara eksplisit oleh program.

BAB III

PENUTUP

Kesimpulan

Xinu menggunakan desain memori yang sederhana dan efisien. Memory image-nya terbagi menjadi empat segmen: text, data, BSS, dan free space. Ketika Xinu berjalan, free space dibagi menjadi dua: stack untuk setiap proses dan heap untuk alokasi dinamis.

- Stack: dialokasikan dari atas ke bawah
- Heap: dialokasikan dari bawah ke atas

Manajemen memori sepenuhnya berada di tangan programmer melalui empat fungsi utama:

- `getstk()` → membuat stack
- `freestk()` → menghapus stack
- `getmem()` → mengambil memori heap
- `freemem()` → mengembalikan memori heap

Karena heap tidak dibersihkan otomatis, programmer wajib melakukan pengelolaan yang benar untuk menghindari memory leak.